



Exercícios de Preparação e Exame — Bases de Dados

Este documento contém os exercícios práticos divididos por temas para consolidar a matéria do guia de estudo, seguidos pelos exercícios reais retirados dos exames anteriores. Tenta resolvê-los primeiro e depois consulta a secção de **Resoluções** no final do documento.



Índice Temático de Exercícios



Enunciados

1. [Conceitos Fundamentais de BD](#)
2. [Arquitetura ANSI/SPARC e Independência de Dados](#)
3. [Modelo Relacional](#)
4. [Álgebra Relacional](#)
5. [SQL – LMD \(Linguagem de Manipulação de Dados\)](#)
6. [SQL – LDD \(Linguagem de Definição de Dados\)](#)
7. [Integridade Referencial](#)
8. [Vistas \(Views\)](#)
9. [Triggers, Stored Procedures e Funções](#)
10. [Normalização](#)
11. [Desenho e Modelação de BD \(Diagramas E/R\)](#)
12. [Data Warehousing](#)
13. [BD Distribuídas e Paralelas](#)
14. [Exercícios Reais de Exames Anteriores](#)



Resoluções

- [Resoluções de Preparação \(Tópicos 1 a 13\)](#)
- [Resoluções de Exame \(Tópico 14\)](#)



Enunciados dos Exercícios

1. Conceitos Fundamentais de BD

Exercício 1.1: LMD Procedimental vs Não-Procedimental

O SQL (Structured Query Language) é classificado como uma linguagem de manipulação de dados (LMD) não-procedimental. No entanto, em SGBDs modernos é frequente utilizar construções procedimentais (como cursores ou blocos de código procedimentais).

1. Explique a diferença teórica fundamental entre uma LMD procedimental e uma LMD não-procedimental.
2. Dê um exemplo prático (em pseudocódigo ou descrição de algoritmo) de como obter a lista de salários superiores a 1000€ usando uma abordagem procedimental, comparando-a com a query SQL equivalente.

Exercício 1.2: Sistemas de Ficheiros vs SGBD

Uma PME gere os seus dados de inventário utilizando folhas de cálculo em Excel partilhadas numa rede local. À medida que a empresa cresce, começam a surgir vários problemas de concorrência e inconsistência.

1. Identifique três limitações graves da gestão de dados por sistema de ficheiros/folhas de cálculo que seriam resolvidas com a migração para um SGBD.
2. Indique uma situação em que a manutenção do sistema de ficheiros seria preferível à instalação de um SGBD.

Exercício 1.3: Componentes de um SGBD e Metadados

O "Catálogo do Sistema" (System Catalog ou Dicionário de Dados) é muitas vezes apelidado de "Metadados".

1. Defina o conceito de Metadados e explique porque é que este repositório é considerado o coração de um SGBD.
2. Explique de que forma os metadados garantem a independência das aplicações face aos dados físicos.

Exercício 1.4: DDL vs DML

Classifique os seguintes comandos SQL como pertencentes à DDL (Linguagem de Definição de Dados) ou à DML (Linguagem de Manipulação de Dados), explicando resumidamente o papel de cada sublinguagem:

1. `ALTER TABLE Cliente ADD Telefone VARCHAR(15);`
2. `UPDATE Produto SET Stock = Stock - 1 WHERE Codigo = 101;`
3. `DROP INDEX idx_nome_cliente;`
4. `INSERT INTO Venda (Data, Total) VALUES (GETDATE(), 120.5);`

2. Arquitetura ANSI/SPARC e Independência de Dados

Exercício 2.1: Níveis da Arquitetura ANSI/SPARC

Considere uma base de dados académica. O Diretor de Curso visualiza a média de notas dos alunos por disciplina; o Administrador da BD vê a estrutura global com tabelas de Alunos, Disciplinas e Inscrições; o Sistema Operativo gere o ficheiro academica.mdf que contém os dados organizados em páginas de 8KB no disco.

1. Associe cada uma destas três visões ao nível correspondente da arquitetura ANSI/SPARC.
2. Explique qual é a utilidade dos mapeamentos (mappings) entre estes três níveis.

Exercício 2.2: Independência Física vs Lógica

A independência de dados divide-se em Independência de Dados Física e Independência de Dados Lógica.

1. Explique a diferença entre estas duas propriedades.
2. Diga qual delas é tipicamente mais difícil de alcançar na prática e porquê.

Exercício 2.3: Cliente-Servidor de 2 e 3 níveis

Compare as arquiteturas cliente-servidor de dois níveis (2-tier) e de três níveis (3-tier) nos seguintes aspetos:

1. Localização das regras de negócio (lógica da aplicação).
2. Escalabilidade e adequação para aplicações Web públicas.

Exercício 2.4: Alteração Física do Armazenamento

Imagine que o administrador da BD decide criar um índice não-agrupado (non-clustered index) sobre a coluna NIF da tabela Cliente para acelerar as pesquisas.

1. Esta alteração ocorre a que nível da arquitetura ANSI/SPARC?
2. Que alteração tem de ser efetuada no código das queries SELECT das aplicações clientes para tirarem partido do índice? Justifique com base no conceito de independência de dados.

3. Modelo Relacional

Exercício 3.1: Grau, Cardinalidade e Atributos

Considere a relação Empregado definida pelo seguinte esquema: Empregado(ID, Nome, DataNascimento, Cargo, NIF, Telefone, ID_Departamento)

1. Qual é o **Grau** desta relação?
2. Se a tabela contiver atualmente registados 150 empregados, qual é a sua **Cardinalidade**?
3. A ordem das linhas no disco físico altera as propriedades lógicas desta relação? Justifique.

Exercício 3.2: Regras de Integridade Relacional

Uma tabela de Inscricoes possui a seguinte estrutura: Inscricoes(NumeroAluno, CodigoDisciplina, AnoLetivo, Nota) A chave primária é composta por (NumeroAluno,

CodigoDisciplina, AnoLetivo). A coluna NumeroAluno referencia a tabela Aluno(Numero) e CodigoDisciplina referencia a tabela Disciplina(Codigo). Diga se as seguintes tentativas de inserção violam alguma regra de integridade relacional, identificando qual:

1. Inserir (NULL, 'BD', '2025/2026', 15).
2. Inserir (12345, 'BD', '2025/2026', 18), sabendo que o aluno 12345 não existe na tabela Aluno.
3. Inserir (10001, 'BD', '2025/2026', 22), sendo o domínio da nota [0, 20].

Exercício 3.3: Chaves Candidatas, Primárias e Superchaves

Considere uma tabela de Cidadãos com as colunas: NIF, NumCC (Cartão de Cidadão), Nome, DataNascimento, Email e Telefone. Sabe-se que NIF, NumCC e Email nunca se repetem entre cidadãos.

1. Identifique as **Chaves Candidatas** desta relação.
2. Dê um exemplo de uma **Superchave** que não seja chave candidata.
3. Como se procede à escolha da **Chave Primária**?

Exercício 3.4: Propriedades de Relações em Tabelas Práticas

Explique por que razão um ficheiro CSV comum contendo linhas duplicadas e colunas onde a mesma linha pode albergar vários valores (ex: uma coluna "Telefones" contendo "912345678; 963332211") **não pode** ser considerado uma Relação no âmbito do Modelo Relacional.

4. Álgebra Relacional

Exercício 4.1: Seleção, Projeção e Diferença (Diferença padrão)

Considere as seguintes relações: Alunos(NumA, Nome, Cidade)Inscritos(NumA, CodD) Escreva a expressão em Álgebra Relacional para listar o "Nome dos alunos de Lisboa que **não** estão inscritos em nenhuma disciplina".

Exercício 4.2: Junções (Natural Join vs Outer Join)

Dadas as relações: Clientes(ID_Cli, Nome)Encomendas(ID_Enc, ID_Cli, Valor) Escreva a expressão em Álgebra Relacional para:

1. Apresentar o Nome do cliente e ID_Enc de todos os clientes que efetuaram encomendas (usando natural join).
2. Apresentar todos os clientes da base de dados e os IDs das respetivas encomendas, garantindo que mesmo os clientes sem encomendas aparecem no resultado (com campos da encomenda a nulo).

Exercício 4.3: Divisão Relacional

Dadas as tabelas: Estudante(NumE, Nome)Inscrito(NumE, CodC)CursoObrigatorio(CodC) Apresente a expressão em Álgebra Relacional para listar os números dos estudantes que estão inscritos em **todos** os cursos obrigatórios.

Exercício 4.4: Compatibilidade de União

Considere as relações: EmpregadosPT(NIF, Nome, Salario)EmpregadosUK(NationalID, Name, Salary, Currency)

1. Explique porque é que a operação $\text{EmpregadosPT} \cup \text{EmpregadosUK}$ não é válida no estado atual das relações.
2. Como resolveria o problema em Álgebra Relacional para poder unir os trabalhadores de ambos os países listando apenas NIF/ID e Nome?

5. SQL – LMD (Linguagem de Manipulação de Dados)

Exercício 5.1: SELECT complexo com JOINS e agrupamentos

Tendo por base o seguinte esquema: $\text{Medicos}(\text{ID_Med}, \text{Nome}, \text{Especialidade})$ $\text{Consultas}(\text{ID_Cons}, \text{Data}, \text{ID_Med}, \text{ID_Pac}, \text{Preco})$ $\text{Pacientes}(\text{ID_Pac}, \text{Nome}, \text{Cidade})$ Escreva a query SQL para obter o "Nome do Médico, a sua especialidade, e a faturação total das suas consultas (soma dos preços) efetuadas em 2025, mas apenas para médicos com faturação total em 2025 superior a 5000€".

Exercício 5.2: Diferença prática entre WHERE e HAVING

Analise a seguinte query SQL que contém um erro e explique qual é o erro e como corrigi-lo:

```
SELECT Categoria, COUNT(ID_Prod) AS Total FROM Produtos WHERE  
COUNT(ID_Prod) > 10 GROUP BY Categoria;
```

Exercício 5.3: Subquery vs Junção (Conversão)

Considere a query que encontra os nomes de clientes que compraram o produto com ID 99 usando uma subquery:

```
SELECT Nome FROM Clientes WHERE ID_Cli IN (      SELECT ID_Cli  
FROM Encomendas      WHERE ID_Prod = 99 );
```

1. Escreva a query equivalente sem usar subqueries (recorrendo a junções).
2. Indique em que situações não é possível reescrever uma subquery como uma junção.

Exercício 5.4: ANY, ALL e EXISTS

Explique a diferença de comportamento lógico entre os seguintes predicados em SQL:

1. $\text{WHERE Preco} > \text{ALL} (\text{SELECT Preco FROM Produtos WHERE Categoria} = \text{'Livros'})$
2. $\text{WHERE Preco} > \text{ANY} (\text{SELECT Preco FROM Produtos WHERE Categoria} = \text{'Livros'})$
3. $\text{WHERE EXISTS} (\text{SELECT 1 FROM Vendas WHERE Vendas.ID_Prod} = \text{Produtos.ID_Prod})$

6. SQL – LDD (Linguagem de Definição de Dados)

Exercício 6.1: Criação de Tabelas com Restrições

Crie o comando CREATE TABLE em SQL para uma tabela de Veiculos com os seguintes requisitos:

- Matrícula: Texto de tamanho fixo de 8 caracteres, chave primária.
- Marca e Modelo: Texto variável, obrigatórios.
- Ano: Inteiro, que deve ser superior a 1900.
- PrecoAluguer: Decimal com 2 casas decimais, com valor por defeito igual a 50.00.

- NumeroQuadro: Texto, deve ser único na tabela.

Exercício 6.2: Alteração de Estrutura de Tabelas

Escreva comandos SQL DDL para:

1. Adicionar uma nova coluna chamada DataAdquisicao (tipo DATE, aceita nulos) à tabela Veiculos criada no Ex. 6.1.
2. Adicionar uma chave estrangeira à tabela Veiculos na coluna ID_Proprietario que referencia a tabela Proprietarios(ID).
3. Alterar a restrição CHECK do Ano para aceitar apenas veículos com Ano superior a 2000.

Exercício 6.3: Criação e Discussão de Índices

Considere uma base de dados com milhões de registos na tabela Vendas(ID, Data, Cliente, Total). As aplicações fazem constantemente duas operações:

1. Pesquisar vendas por Cliente (ex: WHERE Cliente = '123').
 2. Inserir novas vendas a cada segundo.
- Escreva o comando SQL para criar um índice não-agrupado na coluna Cliente.
 - Discuta os prós e contras deste índice para a base de dados.

Exercício 6.4: Remoção de Tabelas (RESTRICT vs CASCADE)

Explique a diferença de comportamento ao executar:

1. DROP TABLE Clientes CASCADE;
2. DROP TABLE Clientes RESTRICT;

7. Integridade Referencial

Exercício 7.1: Operações em Cascata (CASCADE)

Sejam as tabelas: Departamentos(ID_Dep, Nome)Trabalhadores(ID_Trab, Nome, ID_Dep) Sendo ID_Dep em Trabalhadores uma FK para Departamentos configurada com ON DELETE CASCADE e ON UPDATE CASCADE. Imagine o seguinte estado inicial da BD:

Departamentos: (10, 'RH'), (20, 'TI')Trabalhadores: (1, 'Ana', 10), (2, 'Rui', 10), (3, 'Maria', 20)

Mostre o conteúdo final de ambas as tabelas após a execução consecutiva das seguintes instruções SQL:

1. UPDATE Departamentos SET ID_Dep = 15 WHERE ID_Dep = 10;
2. DELETE FROM Departamentos WHERE ID_Dep = 20;

Exercício 7.2: SET NULL e Restrições de Nulos

Considere que pretendemos configurar a relação Curso(ID, Nome) e Aluno(ID_Al, Nome, ID_Curso). Deseja-se que, caso um Curso seja removido, as fichas dos alunos que pertenciam a esse curso se mantenham na BD, mas fiquem com a indicação de que o aluno não tem curso atribuído.

1. Indique a ação referencial a aplicar no ON DELETE da FK de Aluno.

2. Indique que restrição a nível da declaração da coluna ID_Curso na tabela Aluno é obrigatório respeitar para que esta configuração funcione.

Exercício 7.3: Comportamento NO ACTION e RESTRICT

Qual é a diferença teórica e prática entre as ações NO ACTION e RESTRICT aquando da tentativa de apagar um registo pai que possui registos filhos associados?

Exercício 7.4: Chaves Estrangeiras Compostas

Escreva a sintaxe DDL para criar uma tabela Notas com chave estrangeira composta a apontar para Inscricoes(NumAluno, CodDisciplina, AnoLetivo) com propagação em cascata no update e sem ação na deleção.

8. Vistas (Views)

Exercício 8.1: Vistas para Segurança

Uma tabela de Colaboradores contém: ID, Nome, Morada, Telemovel, Cargo, SalarioBase e IBAN. A equipa de limpeza de dados necessita de consultar as moradas e telemóveis para enviar correspondência física, mas não deve ter acesso aos dados financeiros (SalarioBase e IBAN).

1. Crie uma vista que resolva este problema de segurança.
2. Indique como concederia permissões a essa equipa apenas para a vista, retirando acesso à tabela base.

Exercício 8.2: Atualizabilidade de Vistas

Considere as seguintes vistas criadas numa base de dados comercial: Vista_A: CREATE VIEW Vista_A AS SELECT * FROM Artigos WHERE Preco > 50; Vista_B: CREATE VIEW Vista_B AS SELECT Categoria, AVG(Preco) FROM Artigos GROUP BY Categoria; Explique se é possível executar uma operação de UPDATE através de cada uma das vistas, alterando os dados das tabelas base.

Exercício 8.3: Resolução de Vistas (Query Planning)

Dada a vista:

```
CREATE VIEW V_ClientesAtivos AS SELECT ID_Cli, Nome, Cidade FROM Clientes WHERE Ativo = 1;
```

Mostre como o SGBD reescreve internamente a seguinte query efetuada pelo utilizador (mecanismo de resolução de vistas):

```
SELECT Nome FROM V_ClientesAtivos WHERE Cidade = 'Porto';
```

Exercício 8.4: Materialização de Vistas

O que é uma Vista Materializada (Indexed View no SQL Server) e quais os prós e contras da sua utilização comparativamente a uma vista tradicional?

9. Triggers, Stored Procedures e Funções

Exercício 9.1: Trigger para Auditoria (T-SQL)

Escreva o código de um trigger em T-SQL (SQL Server) para a tabela Produtos(ID, Nome, Preco). Sempre que o preço de um produto for alterado (operação UPDATE), o trigger deve

gravar um registo de auditoria na tabela HistoricoPrecos(ID_Prod, PrecoAntigo, PrecoNovo, DataAlteracao, Utilizador). O trigger só deve registar a auditoria se o preço realmente sofrer alteração.

Exercício 9.2: Stored Procedure com Transações

Crie uma Stored Procedure chamada usp_RegistarVenda que receba @ID_Prod INT e @Qtd INT. A SP deve:

1. Iniciar uma transação.
2. Verificar se o produto tem stock suficiente na tabela Produtos.
3. Se sim, deduzir o stock e inserir a venda na tabela Vendas(ID_Prod, Quantidade, Data). Efetuar COMMIT.
4. Se não houver stock suficiente, lançar um erro e efetuar ROLLBACK. Use blocos TRY/CATCH para tratamento de exceções.

Exercício 9.3: Funções Definidas pelo Utilizador (UDF)

Escreva o código SQL para uma função chamada ufn_CalcularDesconto que receba o @PrecoOriginal DECIMAL(10,2) e @AnosCliente INT (anos que o cliente tem de registo na loja). A função deve calcular e devolver o preço com desconto baseado nas seguintes regras:

- Se tem mais de 5 anos de cliente, desconto de 15%.
- Se tem entre 2 e 5 anos (inclusive), desconto de 10%.
- Menos de 2 anos, sem desconto.

Exercício 9.4: Stored Procedures vs Funções

Identifique as três diferenças principais de capacidades e restrições entre uma Stored Procedure e uma UDF (User Defined Function) em SQL.

10. Normalização

Exercício 10.1: Normalização por Atributos (Caso Recibo de Oficina)

Um cliente recebe um recibo simplificado de reparação automóvel com os seguintes atributos: NifOficina, NomeOficina, CodReparacao, Data, Matricula, ModeloCarro, CodPeca, DescricaoPeca, QtdPeca, PrecoPeca, SubtotalPeca, TotalReparacao Utilize o método de normalização por atributos exigido (Passo 0 a Passo 3) para obter as tabelas na 3ª Forma Normal. Apresente as dependências funcionais de cada passo e as definições de cada Forma Normal.

Exercício 10.2: Anomalias de Atualização

Considere uma tabela não normalizada: EmpregadoDepartamento(NIF, Nome, Telefone, CodDep, NomeDep, LocalizacaoDep) Identifique e explique com exemplos práticos as três anomalias de atualização que podem ocorrer nesta tabela.

Exercício 10.3: Decomposição Sem Perdas

O que garante que a divisão de uma tabela desnormalizada em várias tabelas normalizadas não causa perda de informação? Como se designa esta propriedade e como se valida?

Exercício 10.4: Forma Normal de Boyce-Codd (FNBC)

Uma tabela de marcações de consultas armazena: Marcacao(NumUtente, Especialidade, Medico)
Assume-se que:

1. Um utente pode marcar consultas em várias especialidades, mas com apenas um médico por especialidade.
 2. Cada médico trabalha apenas numa especialidade.
- Identifique as dependências funcionais e as chaves candidatas.
 - Explique em que forma normal está a relação (3FN ou FNBC) e proceda à sua normalização se necessário.

11. Desenho e Modelação de BD (Diagramas E/R)

Exercício 11.1: Tipos de Atributos no Diagrama E/R

Desenhe o modelo conceptual (representando as entidades e atributos) para uma entidade Pessoa contendo:

1. NIF (atributo simples e identificador).
2. Nome Completo (composto por Nome Próprio e Apelido).
3. Hobbies (atributo multi-valor).
4. Idade (atributo derivado da Data de Nascimento). Explique como representará graficamente cada um na notação clássica de Chen.

Exercício 11.2: Mapeamento de Relações e Atributos de Ligação

Desenhe o mapeamento lógico para o seguinte relacionamento:

- Um Cliente pode alugar vários Quartos de hotel ao longo do tempo. Um Quarto pode ser alugado por vários Clientes. O ato de alugar possui um atributo DataAluguer e PrecoPago. Apresente as tabelas resultantes, indicando claramente as chaves primárias e estrangeiras.

Exercício 11.3: Especialização, Generalização e Restrições

Considere uma base de dados de uma Rent-a-Car. A classe mãe Veiculo especializa-se em Carro e Mota. Explique o significado das seguintes restrições de especialização e as suas implicações práticas nas tabelas:

1. Restrição de Disjunção: Disjunta (Disjoint) vs Sobreposta (Overlapping).
2. Restrição de Totalidade: Total (Total / Mandatory) vs Parcial (Partial / Optional).

Exercício 11.4: Entidades Fracas (Weak Entities)

O que caracteriza uma Entidade Fraca num modelo E/R? Dê um exemplo prático (entidades, relacionamento e atributos) e explique como é mapeada para o modelo relacional lógico.

12. Data Warehousing

Exercício 12.1: Star Schema vs Snowflake Schema

Compare o esquema em Estrela (Star Schema) com o esquema em Floco de Neve (Snowflake Schema) no desenho de um Data Warehouse. Aborde a normalização das tabelas de dimensão e o impacto na escrita e na leitura dos dados.

Exercício 12.2: Processo ETL e os 5 Fluxos de Dados

O processo de ETL (Extraction, Transformation, and Loading) é crucial num Data Warehouse.

1. Explique as funções de cada fase (E, T, L).
2. Diga a que fluxo de dados da arquitetura de DW (de entre os 5 fluxos: Inflow, Upflow, Downflow, Outflow, Metaflow) corresponde o processo ETL.

Exercício 12.3: Data Warehouse vs Data Mart

Distinga Data Warehouse de Data Mart em termos de:

1. Âmbito organizacional e fontes de dados.
2. Facilidade e custo de implementação.

Exercício 12.4: Características do DW segundo Inmon

Enuncie e explique resumidamente as quatro propriedades fundamentais que caracterizam os dados contidos num Data Warehouse, segundo a definição clássica de William Inmon (1993).

13. BD Distribuídas e Paralelas

Exercício 13.1: Estratégias de Alocação de Dados

Uma multinacional com delegações em Lisboa, Porto e Faro pretende desenhar a sua base de dados distribuída. Compare as vantagens e desvantagens de adotar:

1. Uma estratégia de Replicação Completa (Full Replication).
2. Uma estratégia Particionada (No Replication / Partitioned).

Exercício 13.2: Fragmentação Horizontal vs Vertical

Explique a diferença entre:

1. Fragmentação Horizontal.
2. Fragmentação Vertical. Dê um exemplo de como aplicaria cada uma à tabela Clientes(ID, Nome, Cidade, LimiteCredito) para distribuir dados entre Lisboa e Porto.

Exercício 13.3: Arquiteturas de SGBD Paralelo

No contexto de SGBDs paralelos, distinga as três arquiteturas físicas principais: Shared Memory, Shared Disk e Shared Nothing. Avalie-as quanto à capacidade de expansão (scalability).

Exercício 13.4: Protocolo Two-Phase Commit (2PC)

Explique o funcionamento do protocolo de Commit em Duas Fases (2PC) para garantir a atomicidade das transações distribuídas, detalhando o papel do Coordenador e dos Participantes nas duas fases.

14. Exercícios Reais de Exames Anteriores

Exercício 14.1: Tabela Associativa (Pergunta 8a do exame 2024/2025)

Cenário: Cliente → OrdemFabrico → Produto (M:N com data de entrega). Cria o código em SQL para a tabela associativa resultante.

Exercício 14.2: SQL (Pergunta 8b do exame 2024/2025)

Pergunta: Escreve a query em SQL para apresentar os "Países com mais de 10 clientes que colocaram Ordens de Fabrico no ano de 2024".

Exercício 14.3: Álgebra Relacional (Pergunta 8c do exame 2024/2025)

Pergunta: Escreve a expressão em Álgebra Relacional para apresentar as "Famílias de produtos que NÃO tiveram qualquer ordem de fabrico no 1º trimestre de 2025".

Exercício 14.4: Normalização de Fatura (Pergunta 7 do exame 2024/2025)

Pergunta: Dado um documento típico de fatura de restaurante (com cabeçalho da empresa, cliente, e lista de artigos consumidos), apresenta as tabelas na 3ª Forma Normal e identifica as suas Dependências Funcionais.

Exercício 14.5: Exame 2020/2021 — Álgebra Relacional

Base de dados comum aos Ex. 14.5 e 14.6:

Artigos(Código, Designação, Unidade, Preço) Armazéns(Código, Designação, Localização) Unidades(Código, Designação)
ArtigosArmazéns(Artigos, Armazém, Localização, Stock)
Fornecedores(Número, Nome) FornecedoresArtigos(Fornecedor, Artigo)

1. Apresenta a expressão em Álgebra Relacional para listar os "Armazéns e localizações onde estão armazenados os artigos 'Papel' e 'Tinta'". (1,5 val.)
2. Apresenta a expressão em Álgebra Relacional para indicar "Quais os artigos que NÃO estão no 'Armazém de reciclagem'". (1,5 val.)

Exercício 14.6: Exame 2020/2021 — SQL

Tendo por base a mesma Base de Dados do Ex. 14.5:

1. Crie uma vista que apresente por artigo a quantidade armazenada em cada armazém. (1,5 val.)
2. Para os fornecedores que fornecem mais do que 5 artigos, apresente o número de artigos fornecidos por cada fornecedor com unidade 'Caixa', ordenado decrescentemente. (1,5 val.)

Exercício 14.7: Exame 2022/2023 — Perguntas Teóricas

Lista das 6 perguntas teóricas que saíram neste exame:

1. Conceito de independência de dados e a sua importância. (2 val.)
2. Compare a arquitetura cliente-servidor de dois níveis com a de três-níveis. (2 val.)
3. Diferença entre uma subquery e uma junção. Em que situações não é possível usar uma subquery? (2 val.)

4. O que entende por Integridade Referencial. Identifique as ações das subcláusulas ON DELETE e ON UPDATE. (2 val.)
5. Enuncie as principais abordagens para elaborar o desenho de uma base de dados com múltiplas vistas de utilizadores. (2 val.)
6. Descreva os tipos de atributos num diagrama ER (simples, compostos, multi-valor e derivados). (2 val.)

Exercício 14.8: Exame 2022/2023 — SQL e Álgebra Relacional

Cenário: Empresa de estufas

```
Estufa(codE, descricao, capacidade, cidade) Secção(codigoS, tipo,
estufa)          -- FK: estufa → Estufa Produto(codP, nome,
stock, tipo) Plantação(codP, produto, codS, data_inicio,
data_fim)          -- FK: produto → Produto, codS → Secção
```

(Diagrama E/R: Estufa ← Secção ↑ Plantação → Produto)

1. Apresenta a query em SQL para listar as "Estufas que tiveram mais que 10 plantações do mesmo produto". (3 val.)
2. Apresenta a expressão em Álgebra Relacional para identificar "Quais as secções que NUNCA tiveram plantações". (2 val.)

Exercício 14.9: Exame 2020/2021 — Normalização de Fatura de Vinhos

Pergunta: Desenvolva um esquema conceptual e identifique as dependências funcionais para uma base de dados relacional que suporte a emissão das faturas de uma loja de venda de vinhos. (2 val.)



Resoluções



Resoluções dos Exercícios de Preparação

Resolução do Exercício 1.1 (Conceitos Fundamentais)

1. Diferença Teórica:

- Numa **LMD procedimental**, o utilizador especifica como os dados devem ser obtidos, controlando o fluxo e processamento passo a passo, registo por registo (abordagem "one-record-at-a-time").
- Numa **LMD não-procedimental**, o utilizador apenas especifica que dados quer obter, sem indicar o caminho físico para aceder aos mesmos (abordagem declarativa, "set-at-a-time"). O SGBD encarrega-se de determinar o melhor plano de acesso físico.

2. Exemplo de abordagem:

- Procedimental (pseudocódigo): abrir_ponteiro(cursor_empregados) enquanto cursor_empregados.tem_linhas() fazer linha = cursor_empregados.proxima_linha() se linha.salario > 1000 então guardar_ou_imprimir(linha.nome, linha.salario) fim_se fim_enquanto fechar_ponteiro(cursor_empregados)

- Não-Procedimental (SQL): SELECT nome, salario FROM Empregados WHERE salario > 1000;

Resolução do Exercício 1.2 (Conceitos Fundamentais)

1. Três limitações resolvidas:

- **Redundância e Inconsistência:** No Excel, o mesmo cliente pode estar escrito de várias formas em ficheiros diferentes. Na BD, os dados são centralizados e normalizados.
 - **Acesso Concorrente:** O Excel bloqueia o ficheiro inteiro se alguém estiver a editar. O SGBD gere acessos concorrentes ao nível do registo (linha) recorrendo a locks.
 - **Falta de Integridade:** No Excel pode escrever-se "Texto" numa coluna de preços. A BD impõe restrições de domínio, CHECK e Integridade Referencial.
2. **Quando preferir sistema de ficheiros:** Quando a dimensão do projeto é muito pequena, o custo inicial de instalação e administração do SGBD é proibitivo, a aplicação é de utilizador único, e não há requisitos de partilha concorrente ou de segurança avançada.

Resolução do Exercício 1.3 (Conceitos Fundamentais)

1. **Definição e Importância:** Metadados são dados que descrevem outros dados (como nomes das tabelas, colunas, tipos de dados, chaves e restrições). O Dicionário de Dados é o coração da BD porque todas as operações de validação de queries, permissões de utilizador e escolhas do otimizador de consultas baseiam-se na leitura imediata deste catálogo.
2. **Garantia de Independência:** As aplicações clientes interagem com a BD usando nomes lógicos de tabelas e colunas. Como a BD descreve estas tabelas nos metadados, o SGBD pode traduzir transparentemente os pedidos lógicos para as estruturas físicas de armazenamento. Se mudarmos os ficheiros físicos de disco, apenas os metadados são reajustados, sem tocar nas aplicações.

Resolução do Exercício 1.4 (Conceitos Fundamentais)

1. ALTER TABLE Cliente ADD Telefone VARCHAR(15); → **DDL** (modifica o esquema lógico da base de dados, adicionando um atributo).
2. UPDATE Produto SET Stock = Stock - 1 WHERE Codigo = 101; → **DML** (modifica o valor dos dados armazenados na tabela).
3. DROP INDEX idx_nome_cliente; → **DDL** (remove uma estrutura física/interna de indexação da BD).
4. INSERT INTO Venda (Data, Total) VALUES (GETDATE(), 120.5); → **DML** (insere novos dados numa tabela existente).

Resolução do Exercício 2.1 (ANSI/SPARC)

1. Associação de Níveis:

- Visão do Diretor de Curso (média): **Nível Externo** (vista individual e parcial dos dados).
- Administrador de BD (tabelas Alunos, Disciplinas): **Nível Conceptual** (visão lógica global da BD).

- Ficheiro academica.mdf (páginas de 8KB): **Nível Interno** (visão física do armazenamento).
2. **Utilidade dos Mapeamentos (Mappings):** Garantir a independência de dados. O mapeamento externo-conceitual permite que alterações no esquema conceitual não afetem as vistas externas. O mapeamento conceitual-interno permite alterar a estrutura de armazenamento físico (ex: mover ficheiros ou mudar indexações) sem alterar as tabelas lógicas (esquema conceitual).

Resolução do Exercício 2.2 (ANSI/SPARC)

1. **Diferença:**
 - **Independência Física:** Capacidade de alterar a estrutura física dos ficheiros, discos ou índices sem alterar as tabelas lógicas ou as aplicações.
 - **Independência Lógica:** Capacidade de alterar a estrutura lógica global (esquema conceitual) como adicionar/dividir tabelas sem ter de reescrever as queries das aplicações existentes.
2. **Mais difícil na prática:** A **independência lógica** é muito mais difícil de manter. Se dividirmos uma tabela base em duas por motivos de performance, qualquer query directa a essa tabela vai falhar. A única forma de remediar é criar vistas (views) para simular o comportamento da tabela original, mas vistas com junções têm graves restrições para atualizações de dados.

Resolução do Exercício 2.3 (ANSI/SPARC)

1. **Lógica de negócio:**
 - Numa arquitetura de **2 níveis (2-tier)**, a lógica de negócio reside na máquina cliente (fat client).
 - Numa de **3 níveis (3-tier)**, reside num servidor aplicacional dedicado (Application Server) intermédio.
2. **Escalabilidade e Web:** A arquitetura **3 níveis** é imensamente superior na Web. Permite pooling de conexões (o servidor de aplicação mantém poucas conexões à BD e reutiliza-as de forma concorrente para milhares de utilizadores web), ao passo que em 2 níveis cada utilizador do browser exigiria uma ligação directa aberta à BD, esgotando os recursos do SGBD rapidamente.

Resolução do Exercício 2.4 (ANSI/SPARC)

1. A alteração ocorre no **Nível Interno** (esquema físico de armazenamento de dados).
2. **Nenhuma alteração** é necessária no código SQL. Devido à independência física de dados, o otimizador de consultas do SGBD decide em tempo de execução se deve utilizar o novo índice ou se deve fazer uma leitura total da tabela, sem que o programador precise de instruir a query a usar o índice.

Resolução do Exercício 3.1 (Modelo Relacional)

1. **Grau:** O número de atributos. Como a tabela possui 7 colunas (ID, Nome, DataNascimento, Cargo, NIF, Telefone, ID_Departamento), o grau é **7**.
2. **Cardinalidade:** O número de linhas. Há 150 empregados registados, logo a cardinalidade é **150**.

3. **Não altera.** Segundo a definição de Codd para o Modelo Relacional, uma tabela é um conjunto matemático de tuplos. Por definição de conjuntos, a ordem dos elementos não importa logicamente. Por isso, a alteração da ordem física no disco não afeta as propriedades lógicas da BD.

Resolução do Exercício 3.2 (Modelo Relacional)

1. **Violaria a Integridade da Entidade.** A coluna NumeroAluno é parte integrante da Chave Primária, logo, por regra de integridade de entidade, não pode aceitar valores nulos (NULL).
2. **Violaria a Integridade Referencial.** A coluna NumeroAluno é uma chave estrangeira que referencia Aluno(Numero). Não se pode introduzir um número que não exista previamente na tabela referenciada.
3. **Violaria a Integridade Geral (ou Restrições de Domínio).** A nota 22 excede os limites de validação estipulados para a coluna Nota (intervalo de 0 a 20).

Resolução do Exercício 3.3 (Modelo Relacional)

1. **Chaves Candidatas:** {NIF}, {NumCC}, {Email} (qualquer uma delas identifica unicamente o cidadão de forma mínima).
2. **Superchave:** {NIF, Nome} ou {NumCC, Email} (um superconjunto que engloba uma chave candidata mas não é minimal).
3. **Escolha da Chave Primária:** De entre as candidatas, escolhe-se preferencialmente a mais curta, mais estável (cujo valor não mude ao longo do tempo) e que seja numérica. Neste caso, NIF ou NumCC seriam ideais (evitando o Email, pois o utilizador pode querer alterá-lo com frequência).

Resolução do Exercício 3.4 (Modelo Relacional)

Não se pode considerar uma relação devido a dois fatores:

1. **Falta de atomicidade (Não está na 1FN):** O campo "Telefones" possui dados agrupados e multivalor ("912345678; 963332211") numa única célula, o que viola o requisito de que todos os atributos devem possuir valores atômicos.
2. **Existência de duplicados:** No modelo relacional, todos os tuplos são obrigatoriamente únicos e distinguíveis por intermédio de chaves. O CSV com linhas duplicadas viola esta restrição fundamental de conjuntos.

Resolução do Exercício 4.1 (Álgebra Relacional)

```
-- Filtrar alunos de Lisboa AlunosLx ← σ_Cidade='Lisboa' (Alunos)
-- Obter IDs de todos os alunos de Lisboa IdsLx ←
π_NumA(AlunosLx) -- Obter IDs dos alunos com inscrições
IdsInscritos ← π_NumA(Inscritos) -- Obter IDs de alunos de
Lisboa que NÃO têm inscrições IdsAlvo ← IdsLx - IdsInscritos --
Obter os nomes correspondentes através de junção com a relação
Alunos original Resultado ← π_Nome(Idsalvo ⋈ Alunos)
```

Resolução do Exercício 4.2 (Álgebra Relacional)

1. **Natural Join:** Resultado1 ← π_Nome, ID_Enc(Clientes ⋈ Encomendas)
2. **Left Outer Join:** Resultado2 ← π_Nome, ID_Enc(Clientes ⋈⋈ Encomendas)

Resolução do Exercício 4.3 (Álgebra Relacional)

Para expressar "todos", usamos a operação de divisão ($\div$):

```
-- Isolar colunas necessárias na dividenda RelacaoA ←  $\pi_{NumE, CodC}(Inscrito)$ 
-- Isolar coluna necessária na divisora RelacaoB
←  $\pi_{CodC}(CursoObrigatorio)$ 
-- Efetuar a divisão Resultado ← RelacaoA  $\div$  RelacaoB
```

Resolução do Exercício 4.4 (Álgebra Relacional)

1. **Motivo da Invalidez:** A união exige que as relações sejam **compatíveis com a união**. Isso obriga a que tenham o mesmo grau (número de atributos) e que os domínios das colunas na ordem correspondente coincidam. No caso, EmpregadosPT tem grau 3 e EmpregadosUK tem grau 4.
2. **Resolução:** Devemos projetar apenas as colunas desejadas de cada uma e renomear os cabeçalhos para ficarem coincidentes: $PT_Proj \leftarrow \pi_{NIF, Nome}(EmpregadosPT)$
 $UK_Proj \leftarrow \rho_{(NIF, Nome)}(\pi_{NationalID, Name}(EmpregadosUK))$ Resultado $\leftarrow PT_Proj \cup UK_Proj$

Resolução do Exercício 5.1 (SQL)

```
SELECT m.Nome, m.Especialidade, SUM(c.Preco) AS FaturacaoTotal
FROM Medicos m INNER JOIN Consultas c ON m.ID_Med = c.ID_Med
WHERE c.Data >= '2025-01-01' AND c.Data <= '2025-12-31' GROUP BY
m.ID_Med, m.Nome, m.Especialidade HAVING SUM(c.Preco) > 5000;
```

Resolução do Exercício 5.2 (SQL)

- **Erro:** O SGBD não permite a utilização de funções agregadas (COUNT(ID_Prod)) no WHERE. O WHERE filtra registos individuais antes de estes serem agrupados em memória.
- **Correção:** O filtro de funções agregadas sobre dados já agrupados deve ser efetuado na cláusula HAVING.
- **Query corrigida:** SELECT Categoria, COUNT(ID_Prod) AS Total FROM Produtos GROUP BY Categoria HAVING COUNT(ID_Prod) > 10;

Resolução do Exercício 5.3 (SQL)

1. **Conversão para Join:** SELECT DISTINCT c.Nome FROM Clientes c INNER JOIN Encomendas e ON c.ID_Cli = e.ID_Cli WHERE e.ID_Prod = 99; (Nota: O uso de DISTINCT é necessário para evitar que clientes que tenham feito múltiplas encomendas do produto 99 apareçam duplicados no resultado, emulando perfeitamente a subquery original com IN).
2. **Quando não é possível:** Não é possível reescrever quando lidamos com subqueries escalares utilizadas para comparações agregadas genéricas (ex: WHERE Salario > (SELECT AVG(Salario) FROM Empregados)) ou subqueries correlacionadas em que a cardinalidade dos grupos é fundamental para a computação sem duplicar registos na tabela principal.

Resolução do Exercício 5.4 (SQL)

1. $Preco > ALL (...)$: O produto é selecionado apenas se o seu preço for maior do que o preço de **todos** os produtos na categoria 'Livros' (ou seja, maior do que o livro mais caro).

2. `Preco > ANY (...)`: O produto é selecionado se for mais caro do que pelo menos **um** dos produtos da categoria 'Livros' (ou seja, basta ser mais caro do que o livro mais barato).
3. `EXISTS (...)`: Retorna verdadeiro para a linha atual se a subquery encontrar pelo menos um registo correspondente na tabela de vendas. É um predicado booleano eficiente, pois o SGBD interrompe a avaliação da subquery assim que obtém o primeiro match (semijoin).

Resolução do Exercício 6.1 (SQL)

```
CREATE TABLE Veiculos (
    Matricula CHAR(8),
    Marca VARCHAR(50) NOT NULL,
    Modelo VARCHAR(50) NOT NULL,
    Ano INT CONSTRAINT chk_ano_limite CHECK (Ano > 1900),
    PrecoAluguer DECIMAL(8,2) DEFAULT 50.00,
    NumeroQuadro VARCHAR(30) UNIQUE,
    PRIMARY KEY (Matricula) );
```

Resolução do Exercício 6.2 (SQL)

1. Adicionar coluna: `ALTER TABLE Veiculos ADD DataAdquisicao DATE NULL;`
2. Adicionar FK: `ALTER TABLE Veiculos ADD ID_Proprietario INT; ALTER TABLE Veiculos ADD CONSTRAINT fk_veiculos_proprietarios FOREIGN KEY (ID_Proprietario) REFERENCES Proprietarios(ID);`
3. Alterar constraint CHECK: `ALTER TABLE Veiculos DROP CONSTRAINT chk_ano_limite; ALTER TABLE Veiculos ADD CONSTRAINT chk_ano_limite CHECK (Ano > 2000);`

Resolução do Exercício 6.3 (SQL)

- **Criação de Índice:** `CREATE INDEX idx_vendas_cliente ON Vendas(Cliente);`
- **Discussão:**
 - **Prós:** Acelera exponencialmente a leitura e filtragem de registos por cliente. Evita um Table Scan completo, localizando as vendas associadas ao cliente através de buscas em árvores B (Index Seek).
 - **Contras:** Aumenta o tempo necessário para executar operações de escrita (INSERT, DELETE ou UPDATE do ID do cliente), dado que o SGBD tem de recalculer a árvore do índice em disco. Ocupa também espaço físico adicional.

Resolução do Exercício 6.4 (SQL)

1. `DROP TABLE Clientes CASCADE;` → Elimina a tabela Clientes e todas as restrições que dependam dela noutras tabelas (ex: se Encomendas tiver uma chave estrangeira para Clientes, essa FK é automaticamente removida em cascata).
2. `DROP TABLE Clientes RESTRICT;` → O SGBD rejeita o comando caso existam tabelas ou objetos que dependam da tabela Clientes. O administrador é obrigado a apagar primeiro as referências (ex: FK em Encomendas) antes de conseguir apagar a tabela.

Resolução do Exercício 7.1 (Integridade Referencial)

1. **Após `UPDATE Departamentos SET ID_Dep = 15 WHERE ID_Dep = 10;`** Como a FK tem `ON UPDATE CASCADE`, o ID alterado propaga-se aos trabalhadores 1 e 2:
 - Departamentos: (15, 'RH'), (20, 'TI')

- Trabalhadores: (1, 'Ana', 15), (2, 'Rui', 15), (3, 'Maria', 20)
2. **Após DELETE FROM Departamentos WHERE ID_Dep = 20;** Como a FK tem ON DELETE CASCADE, o departamento 20 é removido e o trabalhador 3 é apagado em cascata:
- Departamentos: (15, 'RH')
 - Trabalhadores: (1, 'Ana', 15), (2, 'Rui', 15)

Resolução do Exercício 7.2 (Integridade Referencial)

1. **Ação Referencial:** ON DELETE SET NULL.
2. **Restrição da coluna:** A coluna ID_Curso na tabela Aluno **não pode** possuir a restrição NOT NULL. Tem de permitir valores nulos para que o SGBD consiga atualizar o campo para NULL no momento em que o curso associado for removido.

Resolução do Exercício 7.3 (Integridade Referencial)

- **RESTRICT:** O SGBD valida a integridade referencial **imediatamente antes** de iniciar a execução do comando de eliminação. Se detetar a presença de registos filhos, aborta a transação imediatamente.
- **NO ACTION:** O SGBD permite iniciar a operação de deleção, mas efetua a validação **no final da instrução SQL** (ou no final da transação, caso seja diferida). Se após a conclusão do ciclo ainda persistirem registos filhos órfãos, é efetuado um ROLLBACK total.

Resolução do Exercício 7.4 (Integridade Referencial)

```
CREATE TABLE Notas (
    ID_Nota INT PRIMARY KEY,
    NumAluno INT,
    CodDisciplina VARCHAR(10),
    AnoLetivo VARCHAR(9),
    ValorNota DECIMAL(4,2),
    CONSTRAINT fk_notas_inscricoes
    FOREIGN KEY (NumAluno, CodDisciplina, AnoLetivo)
    REFERENCES Inscricoes(NumAluno, CodDisciplina, AnoLetivo)
    ON UPDATE CASCADE ON DELETE NO ACTION );
```

Resolução do Exercício 8.1 (Views)

1. **Vista de Segurança:** CREATE VIEW V_Colaboradores_Contacto AS SELECT ID, Nome, Morada, Telemovel, Cargo FROM Colaboradores;
2. **Permissões:** REVOKE ALL PRIVILEGES ON Colaboradores FROM EquipaLimpeza; GRANT SELECT ON V_Colaboradores_Contacto TO EquipaLimpeza;

Resolução do Exercício 8.2 (Views)

- **Vista_A: É atualizável.** Não contém agregações, subqueries, DISTINCT ou GROUP BY e seleciona campos de uma única tabela base.
- **Vista_B: Não é atualizável.** A vista contém cláusulas GROUP BY e funções de agregação (AVG). O SGBD é incapaz de mapear um comando UPDATE na média do grupo para atualizar as colunas dos registos individuais na tabela base.

Resolução do Exercício 8.3 (Views)

O motor de bases de dados funde a consulta do utilizador com a query de definição da vista:

```
SELECT Nome FROM Clientes WHERE Ativo = 1 AND Cidade = 'Porto';
```

Resolução do Exercício 8.4 (Views)

Uma Vista Materializada armazena o resultado da query de definição **fisicamente em disco**, mantendo-se em sincronia automática com as tabelas base.

- **Prós:** Aceleração fantástica na leitura e processamento de consultas OLAP complexas ou agregações pesadas.
- **Contras:** Overhead nas escritas das tabelas base, pois o SGBD necessita de recalcular e atualizar a vista materializada a cada inserção/atualização. Consome espaço em disco.

Resolução do Exercício 9.1 (Triggers/SP/Functions)

```
CREATE TRIGGER trg_AuditoriaPrecoProduto ON Produtos AFTER UPDATE
AS BEGIN      SET NOCOUNT ON;          INSERT INTO HistoricoPrecos
(ID_Prod, PrecoAntigo, PrecoNovo, DataAlteracao, Utilizador)
SELECT i.ID, d.Preco, i.Preco, GETDATE(), SUSER_SNAME()      FROM
inserted i      INNER JOIN deleted d ON i.ID = d.ID      WHERE
i.Preco <> d.Preco; END;
```

Resolução do Exercício 9.2 (Triggers/SP/Functions)

```
CREATE PROCEDURE usp_RegistarVenda      @ID_Prod INT,      @Qtd INT
AS BEGIN      SET NOCOUNT ON;      BEGIN TRANSACTION;
BEGIN TRY      DECLARE @StockAtual INT;      --
Verificar se o produto existe e obter stock      SELECT
@StockAtual = Stock FROM Produtos WHERE ID = @ID_Prod;
IF @StockAtual IS NULL      BEGIN      THROW 50001,
'Produto não registado na base de dados.', 1;      END
IF @StockAtual < @Qtd      BEGIN      THROW 50002,
'Operação abortada: stock insuficiente.', 1;      END
-- Atualizar o stock do produto      UPDATE Produtos
SET Stock = Stock - @Qtd      WHERE ID = @ID_Prod;
-- Registar a venda      INSERT INTO Vendas (ID_Prod,
Quantidade, Data)      VALUES (@ID_Prod, @Qtd, GETDATE());
COMMIT TRANSACTION;      END TRY      BEGIN CATCH      ROLLBACK
TRANSACTION;      DECLARE @Msg NVARCHAR(4000) =
ERROR_MESSAGE();      RAISERROR(@Msg, 16, 1);      END CATCH
END;
```

Resolução do Exercício 9.3 (Triggers/SP/Functions)

```
CREATE FUNCTION ufn_CalcularDesconto (      @PrecoOriginal
DECIMAL(10,2),      @AnosCliente INT ) RETURNS DECIMAL(10,2) AS
BEGIN      DECLARE @PrecoFinal DECIMAL(10,2);      IF
@AnosCliente > 5      SET @PrecoFinal = @PrecoOriginal * 0.85;
ELSE IF @AnosCliente >= 2      SET @PrecoFinal =
@PrecoOriginal * 0.90;      ELSE      SET @PrecoFinal =
@PrecoOriginal;      RETURN @PrecoFinal; END;
```

Resolução do Exercício 9.4 (Triggers/SP/Functions)

1. **Obrigatoriedade de Retorno:** A Função é obrigada a devolver um valor (RETURNS), enquanto o Procedimento pode ou não retornar valores (ou devolver múltiplos recordsets).
2. **Alteração de Dados:** Funções em SQL Server/SGBDs padrão são estritamente de leitura (não podem efetuar comandos DML de escrita INSERT/UPDATE/DELETE nas tabelas base). Stored Procedures não têm essa restrição.
3. **Invocação:** As funções podem ser incorporadas diretamente em expressões SQL normais (ex: no SELECT ou no WHERE), ao passo que os procedimentos têm de ser chamados explicitamente via comando EXEC.

Resolução do Exercício 10.1 (Normalização)

Passo 0: Identificar Atributos com Letras

- A: NifOficina
- B: NomeOficina
- C: CodReparacao
- D: Data
- E: Matricula
- F: ModeloCarro
- G: CodPeca
- H: DescricaoPeca
- I: QtdPeca
- J: PrecoPeca
- K: SubtotalPeca
- L: TotalReparacao

Forma Não Normalizada (UNF): Recibo(A, B, C, D, E, F, G, H, I, J, K, L) Definição UNF: Tabela que contém um ou mais grupos repetidos (aqui, a lista de peças associadas a cada reparação).

Passo 1: 1ª Forma Normal (1FN) Definição 1FN: Uma relação está na 1FN se todas as interseções de linhas e colunas contiverem valores atômicos, não contendo grupos repetidos.

Resolução: Achatamos a tabela. A chave primária que identifica unicamente cada registo é composta por (C, G) (CodReparacao + CodPeca). Recibo_1FN(C, G, A, B, D, E, F, I, J, H, K, L)

Dependências Funcionais (DFs):

- $C \rightarrow A, B, D, E, F, L$
- $G \rightarrow H, J$
- $C, G \rightarrow I, K$
- $A \rightarrow B$
- $E \rightarrow F$

Passo 2: 2ª Forma Normal (2FN) Definição 2FN: Relação na 1FN em que todos os atributos não-chave dependem inteiramente de toda a chave primária (ausência de dependências parciais). Resolução: Extraímos o que depende apenas de partes da PK (C, G):

- **Reparacao**(C, A, B, D, E, F, L) | PK: C
- **Peca**(G, H, J) | PK: G
- **LinhaReparacao**(C, G, I, K) | PK: (C, G) | FK: C → **Reparacao**, G → **Peca**

Identificar Dependências Transitivas na 2FN:

- Na tabela **Reparacao**: C → A e A → B (transitiva C → A → B)
- Na tabela **Reparacao**: C → E e E → F (transitiva C → E → F)

Passo 3: 3ª Forma Normal (3FN) Definição 3FN: Relação na 2FN em que nenhum atributo não-chave depende de forma transitiva da chave primária (todos os determinantes devem ser chaves candidatas). Resolução: Decompomos para eliminar as transitividades causadas por A e E.

Tabelas Finais na 3FN:

- **Oficina**(NifOficina, NomeOficina) | PK: NifOficina
- **Veiculo**(Matricula, ModeloCarro) | PK: Matricula
- **Reparacao**(CodReparacao, Data, NifOficina, Matricula, TotalReparacao) | PK: CodReparacao | FK: NifOficina → **Oficina**, Matricula → **Veiculo**
- **Peca**(CodPeca, DescricaoPeca, PrecoPeca) | PK: CodPeca
- **LinhaReparacao**(CodReparacao, CodPeca, QtdPeca, SubtotalPeca) | PK: (CodReparacao, CodPeca) | FK: CodReparacao → **Reparacao**, CodPeca → **Peca**

Resolução do Exercício 10.2 (Normalização)

1. **Anomalia de Inserção:** Não conseguimos inserir os dados de um novo departamento (ex: sala e nome de um novo departamento de Marketing) antes de termos pelo menos um funcionário contratado para trabalhar lá, porque o campo NIF (chave primária) não aceita valores nulos.
2. **Anomalia de Remoção:** Ao demitirmos o único trabalhador do departamento de Contabilidade, a sua linha na tabela é apagada e perdemos permanentemente a informação física de que esse departamento existia e qual a sua localização.
3. **Anomalia de Modificação (Inconsistência):** Se o departamento de Recursos Humanos for transferido de sala, teremos de atualizar a coluna LocalizacaoDep em todos os empregados do departamento. Se falharmos uma única linha, haverá dados inconsistentes na base de dados.

Resolução do Exercício 10.3 (Normalização)

- A propriedade chama-se **Decomposição Sem Perdas (Lossless-Join)**.
- **Validação:** Garante que, ao aplicar um NATURAL JOIN às tabelas decompostas, recuperamos exatamente a mesma informação e número de registos da tabela desnormalizada original (sem tuplos falsos ou espúrios). Aplica-se o teorema que dita que o atributo comum entre as relações divididas deve ser chave primária (ou superchave) de pelo menos uma delas.

Resolução do Exercício 10.4 (Normalização)

1. DFs e Chaves Candidatas:

- DFs: NumUtente, Especialidade $\rightarrow$ Medico e Medico $\rightarrow$ Especialidade
- Chaves Candidatas: {NumUtente, Especialidade} e {NumUtente, Medico}.

2. Análise de Forma Normal:

- **Na 3FN:** Sim, pois todos os atributos da tabela (NumUtente, Especialidade, Medico) são atributos primos (fazem parte de pelo menos uma chave candidata).
- **Na FNBC:** Não, porque para a DF Medico $\rightarrow$ Especialidade, o determinante Medico não é chave candidata.
- **Normalização para FNBC:**
 - **MedicoEspecialidade**(Medico, Especialidade) | PK: Medico
 - **ConsultaUtente**(NumUtente, Medico) | PK: (NumUtente, Medico) | FK: Medico $\rightarrow$ MedicoEspecialidade

Resolução do Exercício 11.1 (Diagramas E/R)

Notação de Chen:

1. **NIF (Simples/Identificador):** Elipse com o texto "NIF" sublinhado, ligada por uma linha reta à entidade Pessoa.
2. **Nome Completo (Composto):** Elipse "Nome Completo" ligada à entidade, da qual partem duas elipses secundárias ("Nome Próprio" e "Apelido").
3. **Hobbies (Multi-valor):** Elipse com contorno duplo contendo "Hobbies", ligada à entidade.
4. **Idade (Derivado):** Elipse com linha tracejada contendo "Idade", ligada à entidade (calculada em runtime a partir da elipse normal da "Data de Nascimento").

Resolução do Exercício 11.2 (Diagramas E/R)

Trata-se de uma relação N:M.

- **Cliente**(ID_Cliente, Nome, Telemovel) | PK: ID_Cliente
- **Quarto**(NumQuarto, Tipo, PrecoBase) | PK: NumQuarto
- **Aluguer**(ID_Cliente, NumQuarto, DataAluguer, PrecoPago) | PK: (ID_Cliente, NumQuarto, DataAluguer) | FK: ID_Cliente $\rightarrow$ Cliente, NumQuarto $\rightarrow$ Quarto

Resolução do Exercício 11.3 (Diagramas E/R)

1. Disjunção:

- Disjunta (Disjoint): O veículo é Carro ou é Mota, não podendo pertencer a ambas em simultâneo.
- Sobreposta (Overlapping): Um veículo poderia pertencer a ambas em simultâneo (se as classes fossem, por exemplo, Veículo Comercial e Veículo a Diesel).

2. Totalidade:

- **Total (Mandatory):** Todo o veículo genérico na BD tem de pertencer obrigatoriamente a uma subclasse (Carro ou Mota).
- **Parcial (Optional):** Pode haver um veículo na BD que não é Carro nem Mota (ex: Camião), ficando apenas registado na tabela genérica de veículos.

Resolução do Exercício 11.4 (Diagramas E/R)

- **Característica:** Uma entidade fraca não tem atributos que consigam formar uma chave primária por si só. Depende da existência de uma entidade forte (entidade identificadora) e de um relacionamento identificador.
- **Exemplo Lógico:**
 - Empregado(ID_Emp, Nome) | PK: ID_Emp
 - Dependente(ID_Emp, NomeDependente, Parentesco) | PK: (ID_Emp, NomeDependente) | FK: ID_Emp → Empregado (ON DELETE CASCADE)

Resolução do Exercício 12.1 (Data Warehousing)

- **Star Schema:** As tabelas de dimensão encontram-se completamente desnormalizadas (redundantes).
 - Impacto: Leituras muito mais rápidas (OLAP) e queries mais simples porque exige menos junções (JOINS). Ocupa mais espaço em disco.
- **Snowflake Schema:** As tabelas de dimensão encontram-se normalizadas (divididas em sub-tabelas).
 - Impacto: Reduz a redundância de dados e poupa espaço de armazenamento. Contudo, degrada a performance de leitura pois as queries requerem múltiplos JOINS entre dimensões para aceder à informação.

Resolução do Exercício 12.2 (Data Warehousing)

1. **Fases ETL:**
 - **Extraction (Extração):** Captura e leitura dos dados brutos a partir das origens operacionais (ficheiros, bases de dados de produção).
 - **Transformation (Transformação):** Limpeza dos dados, remoção de duplicados, conversão de formatos e aplicação das regras de negócio.
 - **Loading (Carregamento):** Escrita e inserção física dos dados transformados nas tabelas de factos e dimensões do DW.
2. **Fluxo:** Corresponde ao fluxo de dados **Inflow** (entrada).

Resolução do Exercício 12.3 (Data Warehousing)

1. **Âmbito e Fontes:** O Data Warehouse é corporativo e abrange toda a organização, recolhendo dados de todas as áreas de negócio e fontes. O Data Mart é departamental, focando-se apenas num assunto (ex: Finanças).
2. **Custo e Complexidade:** O Data Mart é muito mais barato, simples de desenhar e rápido de implementar. O Data Warehouse é extremamente complexo, requer investimento pesado e pode demorar anos a concretizar.

Resolução do Exercício 12.4 (Data Warehousing)

1. **Orientado a Assuntos:** Dados agrupados por temas centrais do negócio (ex: clientes, vendas) e não por funções operacionais da aplicação.
2. **Integrado:** Unificação rigorosa de codificações e formatos de dados provenientes de fontes heterogêneas.
3. **Variável no Tempo:** Dados armazenam todo o histórico temporal (ex: últimos 5 anos de alterações) em vez de apenas o estado presente.
4. **Não-Volátil:** Dados carregados são apenas de leitura. Não sofrem modificações ou deleções habituais, mantendo a consistência dos relatórios analíticos.

Resolução do Exercício 13.1 (BD Distribuídas)

1. **Replicação Completa:**
 - Vantagens: Elevada tolerância a falhas (se Faro falhar, Lisboa e Porto continuam ativos autónomos) e rapidez na leitura local sem necessidade de tráfego de rede.
 - Desvantagens: Alto custo de armazenamento e lentidão nas escritas (overhead extremo para propagar e sincronizar alterações em todos os sites da rede).
2. **Particionada:**
 - Vantagens: Otimização de armazenamento e alta performance na escrita (cada site insere os seus dados locais rapidamente).
 - Desvantagens: Sem tolerância a falhas (se um nó cair, os seus dados ficam indisponíveis para a rede) e consultas globais lentas devido ao custo de transmissão de dados via rede.

Resolução do Exercício 13.2 (BD Distribuídas)

1. **Fragmentação Horizontal:** Divide as linhas (tuplos) da tabela com base numa condição de filtragem.
 - Exemplo:
 - Lisboa: $RF_1 = \sigma_{Cidade='Lisboa'}(Clientes)$
 - Porto: $RF_2 = \sigma_{Cidade='Porto'}(Clientes)$
2. **Fragmentação Vertical:** Divide as colunas (atributos) da tabela, projetando-as. A PK deve constar em todos os fragmentos.
 - Exemplo:
 - Lisboa: $RF_A = \pi_{ID, Nome, Cidade}(Clientes)$
 - Porto: $RF_B = \pi_{ID, LimiteCredito}(Clientes)$

Resolução do Exercício 13.3 (BD Distribuídas)

- **Shared Memory:** Processadores partilham a mesma memória RAM e discos. Rápido, mas muito fraca escalabilidade (gargalo no acesso à RAM).
- **Shared Disk:** Processadores têm a sua memória privada mas partilham os discos. Escalabilidade intermédia, limitada pelo acesso ao barramento do disco.

- **Shared Nothing:** Processadores têm a sua própria memória RAM e discos autónomos. É o modelo ideal para escalabilidade ilimitada e em larga escala (sistemas Big Data).

Resolução do Exercício 13.4 (BD Distribuídas)

- **Fase 1: Preparação (Prepare):** O nó Coordenador envia um pedido de preparação a todos os nós Participantes. Cada Participante simula a transação até ao ponto de commit, coloca os bloqueios de escrita e vota "Vote_Commit" (se tudo correr bem) ou "Vote_Abort" (se houver erro).
- **Fase 2: Decisão (Commit):** Se todos os Participantes votarem a favor, o Coordenador envia a mensagem "Global_Commit" e todos efetuam a escrita permanente. Se algum votar contra ou falhar o timeout, o Coordenador envia a mensagem "Global_Abort" e todos executam ROLLBACK das operações efetuadas.

Resoluções dos Exercícios de Exame Originais

Resolução do Exercício 14.1

```
CREATE TABLE OrdemFabricoProduto (      Numero INT,          --
FK para OrdemFabrico     CodigoProduto INT,      -- FK para Produto
Quantidade INT,      DataEntrega DATE,      PRIMARY KEY (Numero,
CodigoProduto, DataEntrega),      FOREIGN KEY (Numero) REFERENCES
OrdemFabrico(Numero),      FOREIGN KEY (CodigoProduto) REFERENCES
Produto(CodigoProduto) );
```

Nota: A PK inclui DataEntrega porque o mesmo produto pode aparecer várias vezes na mesma ordem desde que a data de entrega seja diferente.

Resolução do Exercício 14.2

```
SELECT c.País FROM Cliente c INNER JOIN OrdemFabrico o ON
c.CódigoCliente = o.Cliente WHERE YEAR(o.Data) = 2024 GROUP BY
c.País HAVING COUNT(DISTINCT c.CódigoCliente) > 10;
```

Resolução do Exercício 14.3

```
-- Famílias COM ordens no 1º trimestre de 2025: FamiliasComOrdem
← π_Familia(      Produto ⋈ (σ (Data >= '2025-01-01' AND Data <=
'2025-03-31') (      OrdemFabrico ⋈ OrdemFabricoProduto      ))
) -- Todas as famílias: TodasFamilias ← π_Familia(Produto) --
Resultado (Diferença): TodasFamilias - FamiliasComOrdem
```

Resolução do Exercício 14.4

Tabelas Normalizadas:

```
Empresa(NIF_Empresa, Nome, Morada, CodPostal)
-- PK: NIF_Empresa Cliente(NIF_Cliente, Nome)
-- PK: NIF_Cliente Fatura(NumFatura, Data, Hora, NIF_Empresa,
NIF_Cliente,      -- PK: NumFatura      Mesa, Empregado,
Total, MetodoPagamento, ATCUD)      -- FK: NIF_Empresa,
NIF_Cliente Artigo(CodArtigo, Descricao, TaxaIVA, PreçoUnitario)
-- PK: CodArtigo LinhaFatura(NumFatura, CodArtigo, Quantidade,
Subtotal)      -- PK: (NumFatura, CodArtigo)
```

```
-- FK: NumFatura, CodArtigo TaxaIVA(Taxa, Incidencia, ValorIVA,
Total) -- Para detalhe do IVA
```

Dependências Funcionais:

- NumFatura → Data, Hora, NIF_Empresa, NIF_Cliente, Mesa, Empregado, Total, ATCUD
- NIF_Empresa → Nome, Morada, CodPostal
- NIF_Cliente → Nome
- CodArtigo → Descricao, TaxaIVA, PrecoUnitario
- NumFatura, CodArtigo → Quantidade, Subtotal

Resolução do Exercício 14.5

1. Armazéns com Papel e Tinta:

```
-- Armazéns com Papel: ArmazPapel ← π_Armazém, Localização(
ArtigosArmazéns ⋈ (σ_Designação='Papel' (Artigos)) ) -- Armazéns
com Tinta: ArmazTinta ← π_Armazém, Localização(
ArtigosArmazéns ⋈ (σ_Designação='Tinta' (Artigos)) ) -- Armazéns
com AMBOS (interseção): Resultado ← ArmazPapel ∩ ArmazTinta
```

Nota: Se a pergunta pedir "Papel OU Tinta", usar UNIÃO (U) em vez de interseção.

2. Artigos que NÃO estão no Armazém de reciclagem:

```
-- Artigos no armazém de reciclagem: ArtigosReciclagem ←
π_Artigos( ArtigosArmazéns ⋈ (σ_Designação='Armazém de
reciclagem' (Armazéns)) ) -- Todos os artigos: TodosArtigos ←
π_Código(Artigos) -- Resultado (diferença): TodosArtigos -
ArtigosReciclagem
```

Resolução do Exercício 14.6

1. Vista por armazém:

```
CREATE VIEW ArtigosPorArmazem AS SELECT a.Código AS CodArtigo,
a.Designação AS Artigo, az.Código AS CodArmazem,
az.Designação AS Armazem, aa.Stock AS Quantidade FROM
Artigos a INNER JOIN ArtigosArmazéns aa ON a.Código = aa.Artigos
INNER JOIN Armazéns az ON aa.Armazém = az.Código;
```

2. Fornecedores com > 5 artigos em caixas:

```
SELECT f.Nome, COUNT(*) AS NumArtigos FROM Fornecedores f INNER
JOIN FornecedoresArtigos fa ON f.Número = fa.Fornecedor INNER
JOIN Artigos a ON fa.Artigo = a.Código WHERE a.Unidade = 'Cx'
AND f.Número IN ( SELECT fa2.Fornecedor FROM
FornecedoresArtigos fa2 GROUP BY fa2.Fornecedor
HAVING COUNT(*) > 5 ) GROUP BY f.Nome ORDER BY NumArtigos DESC;
```

Nota: A subquery filtra fornecedores com >5 artigos no total; a query exterior conta apenas os de unidade "Caixa".

Resolução do Exercício 14.7

(As respostas detalhadas para estas perguntas teóricas encontram-se estruturadas no [Guia Estudo Exame BD.md](#)).

Resolução do Exercício 14.8

1. SQL: Estufas > 10 plantações mesmo produto:

```
SELECT e.codE, e.descricao FROM Estufa e INNER JOIN Secção s ON
e.codE = s.estufa INNER JOIN Plantação p ON s.codigoS = p.codS
GROUP BY e.codE, e.descricao, p.produto HAVING COUNT(*) > 10;
```

Nota: Agrupamos por estufa E produto, pois queremos contar plantações do **mesmo produto** por estufa. Se a pergunta pedisse "mais que 10 plantações no total", não incluiríamos p.produto no GROUP BY.

2. Álgebra Relacional: Secções SEM plantações:

```
-- Secções COM plantações: SeccoesComPlantacao ←
π_codS(Plantação) -- Todas as secções: TodasSeccoes ←
π_codigoS(Secção) -- Resultado (diferença): TodasSeccoes -
SeccoesComPlantacao
```

Resolução do Exercício 14.9

Esquema conceptual da Fatura de Vinhos:

```
Loja(NIF_Loja, Nome, Morada, CodPostal, Telefone) --
PK: NIF_Loja Cliente(NIF_Cliente, Nome)
-- PK: NIF_Cliente Fatura(NumFatura, Data, Hora, NIF_Loja,
NIF_Cliente, Total) -- PK: NumFatura
-- FK: NIF_Loja, NIF_Cliente Produto(CodProduto, Nome, Categoria,
PrecoUnitario, TaxaIVA) -- PK: CodProduto
LinhaFatura(NumFatura, CodProduto, Quantidade, Subtotal)
-- PK: (NumFatura, CodProduto)
-- FK: NumFatura, CodProduto
```

Dependências Funcionais:

- NumFatura → Data, Hora, NIF_Loja, NIF_Cliente, Total
- NIF_Loja → Nome, Morada, CodPostal, Telefone
- NIF_Cliente → Nome
- CodProduto → Nome, Categoria, PrecoUnitario, TaxaIVA
- NumFatura, CodProduto → Quantidade, Subtotal